COURSE: BCA (4TH SEMESTER)
Operation System -404

Block I: Introduction to Java

1. Explain Java Overview in Detail.

i. Introduction

Java is a high-level, object-oriented, platform-independent programming language developed in 1995 by Sun Microsystems. Later it was acquired by Oracle Corporation.

Java was designed by James Gosling and his team with the main goal of creating a language that could run on multiple platforms without modification.

Java follows the principle:

Write Once, Run Anywhere (WORA)

This means a Java program compiled on one system can run on any system that has JVM installed.

---

ii. Features of Java

✔ Simple

- Syntax similar to C++
- Removed complex features like pointers, operator overloading

✔ Object-Oriented

Java is fully based on OOP principles:

- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

✔ Platform Independent

Java programs are compiled into bytecode.
Bytecode runs on JVM, not directly on OS.

✔ **Secure**

- No direct memory access
- Bytecode verification
- Automatic garbage collection

✔ **Robust**

- Strong exception handling
- Automatic memory management

✔ **Multithreaded**

Supports concurrent execution of multiple threads.

✔ **Distributed**

Supports RMI and networking.

---

iii .Java Program Execution Process

Step 1: Write source code (.java file)
Step 2: Compile using javac → generates bytecode (.class file)
Step 3: JVM executes bytecode
Step 4: Output displayed

Diagram (Required in Exam)

Source Code → Compiler → Bytecode → JVM → Machine Code → Output

---

iv. Applications of Java

- Web applications
- Android applications
- Desktop software
- Enterprise systems
- Banking systems

---

v. Advantages of Java

✔ Portable
✔ Secure
✔ Object-Oriented
✔ Large community support

---

vi. Conclusion

Java is one of the most popular programming languages because of its portability, security, and object-oriented nature. It is widely used in enterprise and mobile application development.

---

2. Explain Difference Between JDK, JRE and JVM in Detail.

i. JVM (Java Virtual Machine)

JVM is an abstract machine that provides runtime environment to execute Java bytecode.

It performs:

- Class loading
- Bytecode verification
- Memory allocation
- Garbage collection
- Execution of code

Without JVM, Java cannot run.

---

ii. JRE (Java Runtime Environment)

JRE = JVM + Supporting Libraries

It provides environment to run Java programs but does not include development tools.

Used by end users.

---

iii. JDK (Java Development Kit)

JDK = JRE + Development Tools

It contains:

- Compiler (javac)
- Debugger
- Documentation tools

Used by developers.

---

iv. Diagram (Important for Exam)

```
JDK
└── JRE
└── JVM
```

---

v. Comparison Table

Feature	JVM	JRE	JDK
Run Program	Yes	Yes	Yes
Compiler	No	No	Yes
Development	No	No	Yes

---

vi. Conclusion

JVM executes bytecode, JRE provides runtime environment, and JDK is used to develop Java applications.

---

3. Explain Internal Details of JVM (Java Virtual Machine) in Detail.

i. Introduction

JVM (Java Virtual Machine) is the core component of Java that provides runtime environment to execute Java byte code. It acts as an intermediary between Java program and operating system.

JVM is platform dependent (different OS → different JVM implementation), but Java bytecode is platform independent.

---

ii. Main Functions of JVM

JVM performs the following tasks:

✔ Loads class files
✔ Verifies bytecode
✔ Allocates memory
✔ Executes program
✔ Performs garbage collection

---

iii. JVM Architecture

JVM architecture mainly consists of:

1. Class Loader Subsystem
2. Runtime Data Areas
3. Execution Engine
4. Native Interface
5. Native Libraries

---

1. Class Loader Subsystem

Class Loader loads .class files into memory.

It performs three main activities:

✓ **Loading**

Reads class file from disk and loads into memory.

✓ **Linking**

- Verification (checks bytecode correctness)
- Preparation (allocates memory for static variables)
- Resolution (replaces symbolic references)

✓ Initialization

Assigns actual values to static variables.

---

2. Runtime Data Areas

These are memory areas used during execution.

✓ Method Area

- Stores class structure
- Static variables
- Method code
- Runtime constant pool

Shared among all threads.

✓ Heap Area

- Stores objects and instance variables
- Garbage collection works here
- Shared among all threads

Example: Student s1 = new Student();
Object stored in Heap.

✓ Stack Area

Each thread has its own stack.

Stores:

- Local variables
- Method calls
- Partial results

When method finishes → stack frame removed.

✓ Program Counter (PC Register)

- Stores address of current instruction.
- Each thread has separate PC register.

✓ Native Method Stack

Used for native (non-Java) methods written in C/C++.

---

iii. Execution Engine

Execution engine executes bytecode.

It contains:

✓ Interpreter

Reads bytecode line by line and executes.

✓ JIT Compiler (Just-In-Time)

Converts bytecode into machine code for faster execution.

JIT improves performance.

---

iv. Java Native Interface (JNI)

Allows Java program to interact with native applications written in C/C++.

---

v. Native Libraries

Libraries written in other languages used by JVM.

JVM Architecture Diagram (Important for Exam)

Structure:

Class Loader
↓
Runtime Data Areas
→ Method Area
→ Heap
→ Stack
→ PC Register
→ Native Stack
↓
Execution Engine
↓
Native Interface

Working of JVM (Step-by-Step)

1. Java source code compiled into bytecode.
2. Class Loader loads bytecode.
3. Bytecode verified.
4. Memory allocated in heap and stack.
5. Execution engine runs code.
6. Garbage collector removes unused objects.

Advantages of JVM

✔ Platform independence
✔ Automatic memory management
✔ Security
✔ Improved performance via JIT

Limitations

✖ Uses more memory
✖ Startup time slightly slow

4. Explain Variables and Data Types in Java in Detail.

1. Introduction

In Java, a variable is a container used to store data values.
Every variable must be declared with a specific data type, which determines the size and type of data it can store.

Java is a strongly typed language, meaning every variable must have a declared data type before use.

---

2. Variable in Java

Definition

A variable is a named memory location that stores a value.

Syntax: **data_type variable_name = value;**

Example: **int age = 20;**
double salary = 25000.50;

Here:

- int and double → Data types
- age and salary → Variable names

---

3. Types of Variables in Java

Java mainly has three types of variables:

---

i. Local Variable

- Declared inside method
- Accessible only within that method
- No default value

Example:

```
void display() {
   int x = 10;  // Local variable
}
```

---

ii. Instance Variable

- Declared inside class but outside method
- Belongs to object
- Each object has its own copy

Example:

```
class Student {
   int roll;  // Instance variable
}
```

---

iii. Static Variable

- Declared using static keyword
- Shared among all objects
- Stored in Method Area

Example:

```
class Student {
   static String college = "ABC";
}
```

---

4. Data Types in Java

Java data types are divided into:

i. Primitive Data Types
ii. Non-Primitive (Reference) Data Types

---

i. Primitive Data Types

Primitive types store simple values directly in memory.

Java has 8 primitive data types:

Data Type	Size	Example
byte	1 byte	byte a = 10;
short	2 bytes	short b = 200;
int	4 bytes	int c = 1000;
long	8 bytes	long d = 10000L;
float	4 bytes	float e = 10.5f;
double	8 bytes	double f = 20.99;
char	2 bytes	char g = 'A';
boolean	1 bit	boolean h = true;

➢ Numeric Types

✔ Integer types → byte, short, int, long
✔ Floating types → float, double

➢ Character Type

Stores single character using Unicode.

Example: `char grade = 'A';`

➢ Boolean Type

Stores only true or false.

Example: `boolean status = true;`

---

2. Non-Primitive Data Types

Also called reference types.

They store reference (address) of object.

Examples:

- String
- Array
- Class
- Interface

- Object

Example: `String name = "Rahul";`

Here, name stores reference of String object.

---

5. Difference Between Primitive and Non-Primitive

Primitive	Non-Primitive
Stores actual value	Stores reference
Fixed size	Size not fixed
Cannot call methods	Can call methods
Lower memory	Higher memory

---

6. Type Casting in Java

Type casting means converting one data type into another.

---

i. Implicit Casting (Widening)

Small → Large data type

Example:

```
int a = 10;
double b = a;
```

Automatic conversion.

---

ii. Explicit Casting (Narrowing)

Large → Small data type

Example:

```
double x = 10.5;
int y = (int) x;
```

Manual conversion required.

---

7. Memory Representation

- Primitive types → stored in stack
- Objects → stored in heap
- Reference → stored in stack

---

8. Advantages of Strong Data Typing

✔ Prevents type mismatch errors
✔ Improves program reliability
✔ Enhances memory management

---

9. Conclusion

Variables and data types are fundamental components of Java programming. Java provides strong typing with primitive and reference data types, ensuring safe and efficient memory usage. Proper understanding of variables and data types is essential for writing error-free Java programs.

---

5. Explain Garbage Collection Mechanism in Java in Detail.

1. Introduction

Garbage Collection (GC) is the automatic process of reclaiming memory by removing unused objects from the heap memory.

In many programming languages like C/C++, programmers manually free memory. But in Java, memory management is automatic and handled by JVM.

This makes Java more secure and robust.

---

2. Need for Garbage Collection

When objects are created using `new` keyword, they are stored in heap memory.

Example:

```
Student s1 = new Student();
```

If the object is no longer referenced by any variable, it becomes useless but still occupies memory.

If such unused objects are not removed:

- Memory leak may occur
- Performance decreases
- Program may crash

To solve this problem, Java uses Garbage Collector.

---

3. What is Garbage?

An object becomes eligible for garbage collection when:

✔ It has no reference
✔ Reference is set to null
✔ Reassigned to another object
✔ Object created inside method and method ends

Example:

```
Student s1 = new Student();
s1 = null;   // Now object becomes eligible for GC
```

---

4. How Garbage Collection Works

Garbage collection works in heap memory.

Step-by-Step Process:

1. Object created in heap.
2. JVM keeps track of references.
3. If no reference points to object → marked as unreachable.
4. Garbage Collector removes it.

5. Memory becomes free.

---

5. Methods Related to Garbage Collection

System.gc()

Used to request JVM to run garbage collector.

```
System.gc();
```

Note: It is only a request, not a guarantee.

---

finalize() Method

Called before object is destroyed.

```
protected void finalize() {
   // cleanup code
}
```

(Note: Modern Java discourages use of finalize.)

---

6. Types of Garbage Collectors (Conceptual)

JVM may use different GC algorithms:

✔ Serial GC
✔ Parallel GC
✔ G1 GC

They differ in performance and memory handling.

---

7. Memory Structure Involved

Garbage collection mainly works in:

✔ Heap Area

- Young Generation

- Old Generation

Objects first go to Young Generation.
If they survive multiple cycles → moved to Old Generation.

---

8. Advantages of Garbage Collection

✔ Automatic memory management
✔ Prevents memory leaks
✔ Reduces programmer burden
✔ Improves security

---

9. Disadvantages

✖ Slight performance overhead
✖ No exact control over memory freeing time

Diagram (Recommended in Exam)

Draw:

Heap Memory
→ Object Created
→ No Reference
→ Garbage Collector
→ Memory Freed

Conclusion

Garbage Collection is an automatic memory management process in Java that removes unused objects from heap memory. It increases reliability, security, and efficiency of Java programs by preventing memory leaks.

---

6. Explain Naming Convention in Java in Detail.

1. Introduction

Naming Convention refers to a set of rules and guidelines used to name variables, methods, classes, packages, and constants in Java.

Although Java compiler does not strictly enforce naming conventions (except syntax rules), following standard conventions improves:

✔ Readability
✔ Maintainability
✔ Professional coding style
✔ Team collaboration

Java follows standard conventions defined by the Java community and documentation guidelines maintained by Oracle Corporation.

---

2. General Rules for Identifiers

An identifier is the name given to:

- Variable
- Method
- Class
- Interface
- Package

Rules:

✔ Must begin with letter, _ (underscore), or $
✔ Cannot start with number
✔ Cannot use reserved keywords (like int, class, if)
✔ Case-sensitive
✔ No spaces allowed

Example (Valid):

```
int totalMarks;
```

Example (Invalid):

```
int 1marks;   // starts with number ✕
```

3. Naming Convention for Different Components

i. Class Naming Convention

✔ Use PascalCase (First letter capital)
✔ Each word starts with capital letter

Example:

```
class StudentRecord
class BankAccount
```

Wrong:

```
class studentrecord ✕
```

---

ii. Interface Naming Convention

✔ Same as class (PascalCase)

Example:

```
interface Runnable
interface Printable
```

---

iii. Method Naming Convention

✔ Use camelCase
✔ First letter small
✔ Verb-based name

Example:

```
calculateTotal()
displayResult()
getMarks()
```

---

iv. Variable Naming Convention

✔ camelCase
✔ Meaningful name
✔ Should describe purpose

Example:

```
int totalMarks;
double accountBalance;
```

Wrong:

```
int x;  // Not meaningful ✕
```

---

v. Constant Naming Convention

✔ All uppercase
✔ Words separated by underscore

Example:

```
static final double PI = 3.14;
static final int MAX_LIMIT = 100;
```

---

vi. Package Naming Convention

✔ All lowercase
✔ Usually reverse domain name

Example:

```
com.company.project
java.util
```

---

4. Why Naming Convention is Important

✔ Makes code readable
✔ Easy debugging
✔ Professional coding standards
✔ Helps in large projects
✔ Avoids confusion

---

5. Example Program Using Naming Convention

```
class StudentRecord {
    static final String COLLEGE_NAME = "ABC College";

    int rollNumber;
    String studentName;
```

```
    void displayDetails() {
        System.out.println(studentName);
    }
}
```

Here:

- Class → StudentRecord
- Constant → COLLEGE_NAME
- Variable → rollNumber
- Method → displayDetails

All follow standard convention.

---

6. Advantages

✔ Improves code clarity
✔ Helps in teamwork
✔ Makes maintenance easier

---

7. Explain OOP Concepts in Java in Detail.

1. Introduction

OOP (Object-Oriented Programming) is a programming paradigm based on objects and classes.

Java is a fully object-oriented programming language (except primitive data types).

OOP helps in:
✔ Code reusability
✔ Data security
✔ Easy maintenance
✔ Real-world modeling

Java implements OOP using classes and objects.

---

2. Four Main Principles of OOP

Java supports 4 main OOP concepts:

i. Encapsulation
ii. Inheritance
iii. Polymorphism
iv. Abstraction

---

i. Encapsulation

Definition

Encapsulation means binding data (variables) and methods together into a single unit (class).

It also means hiding internal data using access modifiers.

➢ How to Achieve Encapsulation

1. Declare variables as private
2. Provide public getter and setter methods

Example:

```
class Student {
    private int marks;

    public void setMarks(int m) {
        marks = m;
    }

    public int getMarks() {
        return marks;
    }
}
```

Advantages

✔ Data security
✔ Controlled access
✔ Improves maintainability

---

ii. Inheritance

Definition

Inheritance is the process by which one class acquires properties and behavior of another class.

Keyword used: extends

Example:

```
class Animal {
    void eat() {
        System.out.println("Eating");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Barking");
    }
}
```

Dog inherits eat() method.

Types of Inheritance

✔ Single
✔ Multilevel
✔ Hierarchical

(Note: Multiple inheritance through classes is not supported in Java.)

Advantages

✔ Code reusability
✔ Reduces redundancy
✔ Improves structure

---

iii. Polymorphism

Definition

Polymorphism means "many forms".

A single method behaves differently in different situations.

Types of Polymorphism

1. Compile-Time Polymorphism

Achieved by method overloading.

```
int add(int a, int b)
int add(int a, int b, int c)
```

---

2. Runtime Polymorphism

Achieved by method overriding.

```
class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    void sound() {
        System.out.println("Bark");
    }
}
```

Advantages

✔ Flexibility
✔ Improves readability
✔ Supports dynamic behavior

---

iv. Abstraction

Definition

Abstraction means hiding implementation details and showing only essential features.

Achieved Using:

✔ Abstract class
✔ Interface

Example:

```
abstract class Shape {
    abstract void draw();
}
```

Only method declaration, no implementation.

Advantages

✔ Reduces complexity
✔ Enhances security
✔ Improves flexibility

Diagram (Recommended in Exam)

*Draw simple diagram:

OOP
→ Encapsulation
→ Inheritance
→ Polymorphism
→ Abstraction

---

4. Real-Life Example

Car:

- Encapsulation → Engine hidden
- Inheritance → ElectricCar extends Car
- Polymorphism → start() behaves differently
- Abstraction → User only drives, doesn't know mechanism

---

8. Explain Object and Class in Java in Detail.

1. Introduction
   Object and Class are the basic building blocks of Object-Oriented Programming in Java. A class is a blueprint, and an object is an instance of that class.
2. Class Definition
   A class is a user-defined data type that contains variables (data members) and methods (member functions).

Syntax:

```
class ClassName {
    // variables
    // methods
}
```

Example:

```
class Student {
    int roll;
    String name;

    void display() {
        System.out.println(roll + " " + name);
    }
}
```

3. Characteristics of a Class
4. It is a logical entity.
5. It does not occupy memory until an object is created.
6. It can contain variables, methods, constructors, blocks, etc.
7. Object Definition
   An object is an instance of a class. It represents a real-world entity and occupies memory in heap.

Syntax:

```
ClassName objectName = new ClassName();
```

Example:

```
Student s1 = new Student();
s1.roll = 101;
s1.name = "Rahul";
s1.display();
```

5. Features of an Object
   Every object has:
6. State – Represented by variables (e.g., roll, name).
7. Behavior – Represented by methods (e.g., display()).
8. Identity – Unique reference in memory.
9. Memory Representation
10. Object is created using the new keyword.
11. Memory is allocated in heap area.
12. Reference variable is stored in stack area.

Diagram Required: Yes.
Draw stack and heap memory diagram showing reference variable in stack and object in heap.

7. Difference Between Class and Object

Class

1. Blueprint of object
2. Logical entity
3. No memory allocated
4. Defined using class keyword

Object

1. Instance of class
2. Physical entity
3. Memory allocated in heap
4. Created using new keyword
5. Example Program

```
class Car {
    String color;
    void start() {
        System.out.println("Car started");
    }
}

public class Main {
    public static void main(String[] args) {
        Car c1 = new Car();
```

```
        c1.color = "Red";
        c1.start();
    }
}
```

9. Advantages
10. Helps in real-world modeling.
11. Improves code organization.
12. Supports encapsulation and abstraction.
13. Conclusion

Class and Object are the foundation of Java programming. A class defines structure and behavior, while objects represent real-world entities created from that class.

---

9. Explain Constructor in Java in Detail.

1. Introduction
   A constructor is a special member of a class that is used to initialize objects. It is automatically called when an object of the class is created.
2. Characteristics of Constructor
3. Name must be same as class name.
4. It does not have any return type (not even void).
5. It is executed automatically when object is created.
6. It is used to initialize instance variables.
7. Syntax

```
class ClassName {
    ClassName() {
        // initialization code
    }
}
```

4. Example

```
class Student {
    int roll;
    String name;

    Student(int r, String n) {
        roll = r;
        name = n;
    }

    void display() {
        System.out.println(roll + " " + name);
    }
}
```

Main class:

```
public class Main {
    public static void main(String[] args) {
        Student s1 = new Student(101, "Rahul");
        s1.display();
    }
}
```

5. Types of Constructors
6. Default Constructor
7. Parameterized Constructor
8. Copy Constructor (user-defined concept in Java)
9. Default Constructor

If no constructor is defined, Java provides a default constructor automatically. It initializes variables with default values.

Example:

```
class Test {
    int x;
}
```

Here x will be initialized to 0 by default constructor.

7. Parameterized Constructor

A constructor that takes arguments to initialize variables.

Example:

```
class Test {
    int x;

    Test(int a) {
        x = a;
    }
}
```

8. Copy Constructor

Java does not provide built-in copy constructor like C++, but we can create one manually.

Example:

```
class Test {
    int x;

    Test(int a) {
```

```
        x = a;
    }

    Test(Test t) {
        x = t.x;
    }
}
```

9. Constructor Overloading

Multiple constructors in same class with different parameters.

Example:

```
class Demo {
    Demo() {
        System.out.println("Default");
    }

    Demo(int a) {
        System.out.println("Parameterized");
    }
}
```

10. Difference Between Constructor and Method

Constructor

1. Used to initialize object.
2. Name same as class.
3. No return type.
4. Called automatically.

Method

1. Used to perform operation.
2. Can have any name.
3. Has return type.
4. Called explicitly.
5. Memory Concept

When object is created using new keyword:

1. Memory allocated in heap.
2. Constructor initializes object.

Diagram Required: Optional but recommended.
Draw object creation in heap and constructor initializing variables.

12. Advantages
13. Ensures object is initialized properly.
14. Improves code reliability.
15. Supports overloading for flexibility.
16. Conclusion

---

10. Explain Static Variable, Static Method and Static Block in Java in Detail.

1. Introduction
   The static keyword in Java is used for memory management. It belongs to the class rather than the object. Static members are shared among all objects of the class.
2. Static Variable

A static variable is a class-level variable shared by all objects.

Characteristics:

1. Declared using static keyword.
2. Only one copy is created for entire class.
3. Stored in Method Area (class area).
4. Accessed using class name.

Syntax:

```
class ClassName {
    static int variableName;
}
```

Example:

```
class Student {
    int roll;
    static String college = "ABC College";

    Student(int r) {
        roll = r;
    }

    void display() {
        System.out.println(roll + " " + college);
    }
}
```

Main method:

```
public class Main {
    public static void main(String[] args) {
        Student s1 = new Student(101);
        Student s2 = new Student(102);

        s1.display();
        s2.display();
    }
}
```

Here college is shared by all students.

Advantages of Static Variable:

1. Saves memory.
2. Used for common properties.
3. Static Method

A static method belongs to the class, not to objects.

Characteristics:

1. Can access only static variables and static methods directly.
2. Cannot use this or super keyword.
3. Called using class name.

Syntax:

```
class ClassName {
    static void methodName() {
        // code
    }
}
```

Example:

```
class Demo {
    static void show() {
        System.out.println("Static Method");
    }
}

public class Main {
    public static void main(String[] args) {
        Demo.show();
    }
}
```

Why main method is static:

1. JVM calls main without creating object.

2. Therefore main must be static.
3. Static Block

A static block is used to initialize static variables.

Characteristics:

1. Executed only once.
2. Executed when class is loaded.
3. Used for complex static initialization.

Syntax:

```
class ClassName {
    static {
        // initialization code
    }
}
```

Example:

```
class Test {
    static int x;

    static {
        x = 100;
        System.out.println("Static Block Executed");
    }
}
```

Static block runs before main method.

5. Order of Execution
6. Static block
7. Main method
8. Constructor (when object created)
9. Difference Between Static and Non-Static

Static

1. Belongs to class
2. Shared memory
3. Access using class name
4. Loaded once

Non-Static

1. Belongs to object
2. Separate copy per object

3. Access using object reference
4. Loaded when object created
5. Memory Representation
6. Static members stored in Method Area.
7. Instance variables stored in Heap.
8. Local variables stored in Stack.

Diagram Required: Yes (optional but good for 10 marks).
Draw Method Area, Heap, and Stack.

8. Advantages
9. Efficient memory usage.
10. Useful for common data.
11. Improves performance.
12. Conclusion

Static variable, method, and block are class-level members in Java. They help in memory optimization and allow shared access across all objects of the class.

---

11. Explain this Keyword in Java in Detail.

1. Introduction
   The keyword in Java is a reference variable that refers to the current object of the class. It is used inside instance methods or constructors to refer to the object that is currently invoking the method or constructor.
2. Need of this Keyword

Sometimes local variables (inside method or constructor) and instance variables (class variables) have same name. In such cases, this keyword is used to differentiate between instance variable and local variable.

3. Basic Example

```
class Student {
    int roll;

    Student(int roll) {
        this.roll = roll;
    }

    void display() {
        System.out.println(roll);
    }
}
```

Explanation:

1. roll inside constructor is local variable.
2. this.roll refers to instance variable of the class.
3. this distinguishes class variable from local variable.
4. Main Uses of this Keyword
5. To refer current class instance variable.
6. To invoke current class method.
7. To invoke current class constructor.
8. To pass current object as argument.
9. To return current object from method.
10. Use 1: Referring Instance Variable

```
class Test {
    int x;

    Test(int x) {
        this.x = x;
    }
}
```

This avoids confusion between local and instance variable.

6. Use 2: Invoking Current Class Method

```
class Demo {
    void show() {
        System.out.println("Show method");
    }

    void display() {
        this.show();
    }
}
```

Here this.show() calls current class method.

7. Use 3: Invoking Current Class Constructor

This is called Constructor Chaining.

---

12. Explain Method Overloading in Java in Detail.

1. Introduction
   Method Overloading is a feature of Java where multiple methods have the same name but different parameters in the same class. It is an example of compile-time polymorphism.
2. Conditions for Method Overloading
3. Method name must be same.
4. Parameter list must be different (number or type or order).

5. Return type may or may not be same, but only changing return type is not sufficient.
6. Example

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }

    double add(double a, double b) {
        return a + b;
    }
}
```

4. Types of Changes Allowed
5. Change in number of parameters.
6. Change in data type.
7. Change in order of parameters.
8. Advantages
9. Improves readability.
10. Increases flexibility.
11. Supports compile-time polymorphism.
12. Conclusion
    Method overloading allows same method name to perform different tasks based on parameters

---

13. Explain Inheritance in Java in Detail.

1. Introduction
   Inheritance is the process by which one class acquires properties and behavior of another class using extends keyword.
2. Syntax

```
class Parent {
    void show() {
        System.out.println("Parent class");
    }
}

class Child extends Parent {
    void display() {
        System.out.println("Child class");
    }
}
```

3. Types of Inheritance
4. Single
5. Multilevel
6. Hierarchical

Note: Java does not support multiple inheritance through classes.

4. Advantages
5. Code reusability.
6. Reduces redundancy.
7. Easy maintenance.

Diagram Required: Yes.
Draw parent class and child class relationship.

5. Conclusion
   Inheritance promotes reusability and hierarchical classification.

---

14. Explain Method Overriding in Detail.

1. Introduction: Method overriding occurs when a child class provides specific implementation of a method already defined in parent class.
2. Conditions
3. Same method name.
4. Same parameters.
5. Inheritance required.
6. Access modifier cannot be more restrictive.
7. Example

```
class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    void sound() {
        System.out.println("Bark");
    }
}
```

4. Advantage
5. Supports runtime polymorphism.
6. Allows dynamic method dispatch.

---

15. Explain super Keyword in Java in Detail.

1. Introduction
   The super keyword in Java is a reference variable used to refer to the immediate parent class object. It is mainly used in inheritance when a child class wants to access members (variables, methods, constructor) of its parent class.

When a class extends another class, the child class inherits the properties of the parent class. If there is ambiguity between parent and child members, super is used to differentiate them.

2. Need of super Keyword

When:

1. Parent and child class have same variable name.
2. Parent and child class have same method name.
3. Child class wants to call parent class constructor.

In these cases, super is used to access parent class members.

3. Uses of super Keyword
4. To access parent class variable.
5. To call parent class method.
6. To call parent class constructor.
7. Use 1: Accessing Parent Class Variable

When child class has variable with same name as parent class, super is used to access parent variable.

Example:

```
class Parent {
    int x = 10;
}

class Child extends Parent {
    int x = 20;

    void display() {
        System.out.println("Child x = " + x);
        System.out.println("Parent x = " + super.x);
    }
}
```

Explanation:

1. x refers to child class variable.
2. super.x refers to parent class variable.

3. Use 2: Calling Parent Class Method

If child class overrides a method, and we want to call parent version also, super is used.

Example:

```
class Parent {
    void show() {
        System.out.println("Parent method");
    }
}

class Child extends Parent {
    void show() {
        System.out.println("Child method");
        super.show();
    }
}
```

Explanation:

1. Child method overrides parent method.
2. super.show() calls parent class method.
3. Use 3: Calling Parent Class Constructor

When an object of child class is created, parent class constructor is automatically called. We can explicitly call it using super().

Example:

```
class Parent {
    Parent() {
        System.out.println("Parent constructor");
    }
}

class Child extends Parent {
    Child() {
        super();
        System.out.println("Child constructor");
    }
}
```

Important Rules:

1. super() must be the first statement in constructor.
2. If not written explicitly, Java automatically calls default parent constructor.
3. Order of Constructor Execution
4. Parent class constructor
5. Child class constructor

16. Explain final Keyword in Java in Detail.

1. Introduction
   The final keyword in Java is a non-access modifier used to restrict modification. It can be applied to variables, methods, and classes. Once something is declared as final, it cannot be changed further.

The purpose of final is to provide security, prevent modification, and ensure consistency in programs.

2. Uses of final Keyword

The final keyword can be used with:

1. Variable
2. Method
3. Class
4. final Variable

A final variable becomes constant. Once a value is assigned, it cannot be changed.

Example:

```
class Test {
    final int x = 10;

    void show() {
        // x = 20;  // Error: cannot change value
        System.out.println(x);
    }
}
```

Q17. Explain Runtime Polymorphism in Java in Detail.

1. Introduction

Polymorphism means “many forms”. In Java, polymorphism allows one method to behave differently based on the object that calls it.

Runtime Polymorphism is a type of polymorphism where the method to be executed is determined at runtime, not at compile time.

It is achieved using:

1. Inheritance

2. Method Overriding
3. Parent class reference pointing to child class object
4. Definition

Runtime Polymorphism is achieved when a parent class reference variable refers to a child class object and the overridden method is called based on the object at runtime.

This concept is also called Dynamic Method Dispatch.

3. Why It Is Called Runtime Polymorphism

Because:

1. The method call is resolved at runtime.
2. The decision of which method to execute depends on object type.
3. Basic Example

```
class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }
}

class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}

class Cat extends Animal {
    void sound() {
        System.out.println("Cat meows");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal a;

        a = new Dog();
        a.sound();

        a = new Cat();
        a.sound();
    }
}
```

Explanation:

1. Animal is parent class.
2. Dog and Cat override sound() method.
3. Parent reference a refers to different child objects.

4. Method executed depends on object, not reference type.
5. Key Conditions for Runtime Polymorphism
6. Inheritance must exist.
7. Method must be overridden.
8. Parent reference must refer to child object.

Example:

Animal a = new Dog();

This is valid because Dog is a type of Animal.

6. Dynamic Method Dispatch

Dynamic method dispatch is the mechanism by which a call to an overridden method is resolved at runtime.

---

18. Explain Abstract Class in Java in Detail.

1. Introduction

An abstract class in Java is a class that is declared using the abstract keyword. It cannot be instantiated (object cannot be created directly). It is mainly used to provide a base for subclasses and to achieve abstraction.

Abstract class can contain:

1. Abstract methods (without body)
2. Concrete methods (with body)
3. Constructors
4. Static methods
5. Final methods
6. Definition

An abstract class is a class that contains at least one abstract method and is declared using the abstract keyword.

3. Abstract Method

An abstract method is a method without implementation (without body). It only contains method declaration.

Syntax:

abstract returnType methodName();

Example:

abstract class Animal {
abstract void sound();
}

4. Example of Abstract Class

```
abstract class Animal {
    abstract void sound();   // abstract method

    void eat() {             // concrete method
        System.out.println("Animal eats food");
    }
}

class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal a = new Dog();
        a.sound();
        a.eat();
    }
}
```

---

19. Explain Wrapper Classes in Java in Detail.

1. Introduction

Wrapper classes in Java are used to convert primitive data types into objects. Since Java is a purely object-oriented language, many utilities and frameworks work only with objects. Primitive data types such as int, char, double etc. are not objects. Therefore, Java provides corresponding wrapper classes for each primitive type.

2. Definition

A wrapper class is a predefined class in Java that wraps (encloses) a primitive data type into an object form.

3. Primitive Types and Corresponding Wrapper Classes

Primitive Type → Wrapper Class

1. byte → Byte
2. short → Short
3. int → Integer
4. long → Long
5. float → Float
6. double → Double
7. char → Character
8. boolean → Boolean

These classes are part of the java.lang package of Java.

4. Need of Wrapper Classes

Wrapper classes are required because:

i. Collections Framework (like ArrayList, HashSet) stores only objects, not primitive types.
ii. They provide useful utility methods for conversion.
iii. Required in multithreading and synchronization.
iv. Enable object manipulation of primitive data.

Example: ArrayList cannot store int directly, it stores Integer objects.

5. Conversion Methods

Wrapper classes provide methods for conversion:

i. valueOf() – converts primitive to wrapper object.
ii. parseInt(), parseDouble() – converts String to primitive.
iii. intValue(), doubleValue() – converts wrapper object to primitive.
iv. toString() – converts value to String.

Example:

```
int x = 10;
Integer obj = Integer.valueOf(x);
int y = obj.intValue();
```

6. Autoboxing and Unboxing

Autoboxing
Automatic conversion of primitive into wrapper object.

Example:
```
Integer obj = 10;
```

Unboxing Automatic conversion of wrapper object into primitive.

Example: int x = obj;

These features were introduced to simplify programming.

7. Memory Concept
8. Primitive variables are stored in stack memory.
9. Wrapper objects are stored in heap memory.
10. Wrapper classes consume more memory compared to primitives.
11. Advantages
12. Allows use of primitive data in object form.
13. Essential for Java Collections Framework.
14. Provides many built-in utility methods.
15. Improves flexibility in programming.
16. Disadvantages
17. More memory usage.
18. Slightly slower performance compared to primitive types.
19. Conclusion

Wrapper classes are an important feature of Java that convert primitive data types into objects. They are mainly used in collections, type conversion, and object-based operations. Autoboxing and unboxing make wrapper classes easy and efficient to use.

---

20. Explain String, StringBuffer and StringBuilder in Java

1. Introduction

In Java, String is a class used to represent sequence of characters. Java provides three main classes to handle strings:

i. String
ii. StringBuffer
iii. StringBuilder

All three are available in the java.lang package but they differ in mutability and performance.

2. String Class

A String is an immutable class in Java. Once a String object is created, its value cannot be changed.

Example: String s = "Hello";
s = s + " World";

In this case, a new object is created instead of modifying the existing one.

Important Points:

i. Immutable (cannot be modified).
ii. Stored in String Constant Pool.
iii. Thread-safe due to immutability.

Advantages:

i. Security.
ii. Safe for multithreading.
iii. Used widely in Java programs.

Disadvantage:

i. Slower when frequent modifications are required.
ii. StringBuffer Class

3. StringBuffer is a mutable class. It means the value of object can be modified without creating a new object.

Example: StringBuffer sb = new StringBuffer("Hello");
sb.append(" World");

Features:

i. Mutable (modifiable).
ii. Thread-safe (methods are synchronized).
iii. Slower than StringBuilder due to synchronization.

Used when multiple threads are working on the same string.

4. StringBuilder Class

StringBuilder is also a mutable class like StringBuffer, but it is not thread-safe.

Example: StringBuilder sb = new StringBuilder("Hello");
sb.append(" World");

Features:

i. Mutable.

ii. Not synchronized.
iii. Faster than StringBuffer.

Used in single-threaded environments for better performance.

5. Difference Between String, StringBuffer and StringBuilder
6. Mutability
   String → Immutable
   StringBuffer → Mutable
   StringBuilder → Mutable
7. Thread Safety
   String → Yes
   StringBuffer → Yes
   StringBuilder → No
8. Performance
   String → Slow (creates new objects)
   StringBuffer → Moderate
   StringBuilder → Fast
9. Memory Concept
10. String objects are stored in String Constant Pool.
11. StringBuffer and StringBuilder objects are stored in Heap memory.
12. When to Use
13. Use String when data does not change frequently.
14. Use StringBuffer in multithreaded programs.
15. Use StringBuilder in single-threaded programs for better performance.
16. Conclusion

String, StringBuffer, and StringBuilder are used to handle character data in Java. String is immutable and safe, while StringBuffer and StringBuilder are mutable. StringBuffer is thread-safe but slower, whereas StringBuilder is faster but not thread-safe.

Block II: Exception Handling

Q1. What is Exception Handling in Java?

Introduction

Exception handling is a mechanism in Java used to handle runtime errors so that the program does not terminate abnormally. It allows the program to continue execution even after an error occurs.

Example

```
try {
int a = 10;
int b = 0;
int c = a / b;
}
catch (ArithmeticException e) {
System.out.println("Cannot divide by zero");
}
```

Advantages

1. Prevents abnormal termination of program.
2. Maintains smooth execution.
3. Improves reliability of software.
4. Separates error-handling code from normal code.

Disadvantages

1. Increases code length.
2. Improper handling may hide actual errors.

Q2. Explain Types of Exceptions in Java.

Introduction

In Java, exceptions are classified into different types based on when and how they occur. This classification helps in proper handling of errors.

Example

```
int a = 10/0;
```

This causes ArithmeticException (Unchecked Exception).

FileReader fr = new FileReader("abc.txt");
This may cause IOException (Checked Exception).

Main Types

1. Checked Exceptions
   These are checked at compile time and must be handled.
   Example: IOException, SQLException

2. Unchecked Exceptions
   These occur at runtime and are not checked at compile time.
   Example: ArithmeticException, NullPointerException
3. Errors
   These are serious problems and cannot be handled normally.
   Example: OutOfMemoryError

Advantages

1. Helps in better error classification.
2. Forces programmer to handle important exceptions.
3. Improves program safety.

Disadvantages

1. Too many checked exceptions make code lengthy.
2. Sometimes handling becomes complex.

Q3. Explain try and catch Block in Java.

Introduction

The try and catch blocks are used in Java to handle exceptions. The try block contains risky code that may cause an exception, and the catch block handles the exception if it occurs.

Example:

```
try {
int a = 10;
int b = 0;
int c = a / b;
}
catch (ArithmeticException e) {
System.out.println("Error: Division by zero");
}
```

Advantages

1. Prevents program from crashing.
2. Maintains normal program flow.
3. Allows specific handling of errors.
4. Improves readability by separating error-handling code.

Disadvantages

1. Makes code longer.
2. If not used properly, may hide actual errors.
3. Too many catch blocks reduce clarity.

Q4. Explain Multiple Catch Block in Java.

Introduction

Multiple catch blocks are used when a single try block can throw more than one type of exception. It allows handling different exceptions separately.

Example

```
try {
int arr[] = new int[5];
arr[10] = 50;
int a = 10 / 0;
}
catch (ArrayIndexOutOfBoundsException e) {
System.out.println("Array Index Error");
}
catch (ArithmeticException e) {
System.out.println("Arithmetic Error");
}
```

Advantages

1. Provides specific handling for different exceptions.
2. Improves clarity of error messages.
3. Makes program more reliable.

Disadvantages

1. Order of catch blocks must be correct (specific before general).
2. Too many catch blocks can make code complex.

Q5. Explain Nested try Block.

Introduction

Nested try block means placing a try block inside another try block. It is used when a part of code inside a try block needs separate exception handling. This provides layered and more detailed error handling.

Example

```
try {
try {
int a = 10/0;
}
catch (ArithmeticException e) {
System.out.println("Inner catch");
}
}
catch (Exception e) {
System.out.println("Outer catch");
}
```

Advantages

1. Provides layered exception handling.
2. Allows handling specific part separately.
3. Improves control over complex code.
4. Increases program flexibility.

Disadvantages

1. Makes code more complex.
2. Harder to read and maintain.
3. May confuse beginners.

Q6. Explain finally Block.

Introduction

The finally block in Java is used to execute important code such as closing resources, releasing memory, or closing database connections. It always executes whether an exception occurs or not. The finally block is written after try and catch blocks.

Example

```
try {
int a = 10/0;
}
catch (ArithmeticException e) {
System.out.println("Exception handled");
}
finally {
System.out.println("This block always executes");
}
```

Advantages

1. Ensures resource cleanup.
2. Always executes regardless of exception.
3. Prevents memory leaks.
4. Improves reliability of program.

Disadvantages

1. Makes code slightly longer.
2. Sometimes unnecessary if no resource used.
3. Cannot prevent program termination in serious errors.

Q7. Explain throw Keyword.

Introduction

The throw keyword is used to explicitly throw an exception in Java. It allows the programmer to manually generate an exception when certain conditions are not satisfied.

Example

```
if(age < 18) {
throw new ArithmeticException("Not eligible");
}
```

Advantages

1. Allows custom error generation.
2. Improves validation of inputs.
3. Makes program more secure.
4. Provides meaningful error messages.

Disadvantages

1. Improper use may disturb program flow.
2. Can make logic complex.
3. Requires proper handling.

Q8. Explain throws Keyword.

Introduction

The throws keyword is used in method declaration to specify that the method may throw one or more exceptions. It informs the caller of the method that it must handle or further declare the exception.

Example

```
void readFile() throws IOException {
FileReader fr = new FileReader("abc.txt");
}
```

Advantages

1. Provides clear information about possible exceptions.
2. Supports exception propagation.
3. Improves program transparency.
4. Maintains structured handling.

Disadvantages

1. Passes responsibility to caller.
2. Can increase dependency between methods.
3. May complicate method declarations.

Q9. Explain Exception Propagation.

Introduction

Exception propagation is the process of passing an exception from one method to another method in the call stack. If a method does not handle an exception, it is forwarded to the calling method.

Example

```
void m1() {
m2();
}

void m2() {
int a = 10/0;
}
```

Advantages

1. Centralized handling of exceptions.
2. Reduces duplicate handling code.
3. Improves code organization.
4. Makes large applications manageable.

Disadvantages

1. If not handled anywhere, program terminates.
2. Hard to trace in large programs.
3. Can increase complexity.

Q10. Explain Exception Handling with Method Overriding.

Introduction

In Java, when a method is overridden in a child class, exception handling rules must be followed. The child class can throw the same exception, a subclass exception, or no exception, but cannot throw a broader exception than the parent method.

Example

```
class Parent {
void show() throws IOException { }
}
```

```
class Child extends Parent {
void show() throws IOException { }
}
```

Advantages

1. Maintains compatibility between parent and child.
2. Ensures safe inheritance.
3. Supports polymorphism.
4. Prevents unexpected runtime issues.

Disadvantages

1. Strict rules must be followed.
2. Can confuse beginners.
3. Reduces flexibility in some cases.

Q11. Explain Custom Exception.

Introduction

A custom exception is a user-defined exception created by extending the Exception class. It is used when built-in exceptions are not sufficient to represent a specific error.

Example

```
class MyException extends Exception {
MyException(String msg) {
super(msg);
}
}
```

Advantages

1. Provides meaningful error messages.
2. Improves program clarity.
3. Useful for business logic validation.
4. Makes application more structured.

Disadvantages

1. Requires additional class creation.
2. Increases code size.

Needs proper documentation.

Q12. Difference Between Checked and Unchecked Exceptions.

Introduction

Checked exceptions are checked at compile time and must be handled, while unchecked exceptions occur at runtime and are not compulsory to handle.

Example

Checked: IOException
Unchecked: ArithmeticException

Advantages

1. Improves safety.
2. Ensures important errors are handled.
3. Makes system reliable.
4. Encourages disciplined coding.

Disadvantages

Makes code lengthy.

1. Handling may become repetitive.
2. Reduces readability in large programs.

Q13. Difference Between throw and throws.

Introduction

Throw is used to explicitly generate an exception, while throws is used to declare exceptions in method definition.

Example

```
throw new ArithmeticException();
```

void test() throws IOException { }

Advantages

1. Structured exception management.
2. Improves clarity of program.
3. Supports propagation mechanism.

Disadvantages

1. Can create confusion.
2. May complicate method signatures.
3. Improper use affects flow.

Q14. What is ArithmeticException?

Introduction

ArithmeticException occurs when an illegal arithmetic operation is performed, such as division by zero.

Example

```
int a = 10/0;
```

Advantages

1. Identifies arithmetic errors.
2. Prevents incorrect calculations.
3. Helps debugging.

Disadvantages

1. Occurs due to lack of validation.
2. Can crash program if not handled.
3. Common runtime mistake.

Q15. What is NullPointerException?

Introduction

NullPointerException occurs when a program tries to use an object reference that has not been initialized.

Example

String s = null;
s.length();

Advantages

1. Detects null reference usage.
2. Helps identify programming mistakes.
3. Improves code correction.

Disadvantages

1. Very common runtime error.
2. Hard to trace in large applications.
3. Causes program termination if not handled.

Q16. What is ArrayIndexOutOfBoundsException?

Introduction

This exception occurs when an array index exceeds its declared size.

Example

```
int arr[] = new int[5];
arr[10] = 20;
```

Advantages

1. Prevents invalid memory access.
2. Ensures array safety.
3. Detects indexing mistakes.

Disadvantages

1. Occurs due to poor indexing logic.
2. Can terminate program.

3. Common beginner error.

Q17. What is IOException?

Introduction

IOException occurs during input-output operations such as reading or writing files.

Example

```
FileReader fr = new FileReader("abc.txt");
```

Advantages

1. Detects file-related errors.
2. Ensures safe file handling.
3. Prevents data corruption.

Disadvantages

1. Must be handled compulsorily.
2. Makes file-handling code lengthy.
3. Requires proper resource closing.

Q18. What is SQLException?

Introduction

SQLException occurs when there is an error while interacting with a database.

Example

```
Connection con = DriverManager.getConnection(...);
```

Advantages

1. Detects database errors.
2. Ensures safe database transactions.
3. Improves application reliability.

Disadvantages

1. Complex to handle.
2. Requires database knowledge.
3. Makes code lengthy.

Q19. What is StackOverflowError?

Introduction

StackOverflowError occurs when stack memory is full, usually due to infinite recursion.

Example

```
void test() {
test();
}
```

Advantages

1. Indicates recursion issue.
2. Helps detect logic errors.
3. Prevents uncontrolled memory usage.

Disadvantages

1. Program crashes.
2. Hard to recover from.
3. Caused by poor recursion design.

Q20. What is OutOfMemoryError?

Introduction

OutOfMemoryError occurs when heap memory is full and no more objects can be created.

Example

Creating too many objects inside loop without releasing memory.

Advantages

1. Detects memory problems.
2. Prevents system crash.
3. Indicates need for memory optimization.

Disadvantages

1. Serious error.
2. Hard to recover.
3. May terminate program immediately.

Block III: Multithreading

Q1. What is Multithreading in Java?

Introduction

Multithreading is a process of executing two or more threads simultaneously within a single program. A thread is the smallest unit of execution inside a process. In Java, multithreading allows multiple tasks to run concurrently, which improves the performance and responsiveness of applications. It is mainly used in applications where multiple operations need to be performed at the same time, such as web servers, games, animations, and real-time systems.

Java provides built-in support for multithreading through the Thread class and Runnable interface. All threads share the same memory space but execute independently.

Explanation / Working

1. A thread is created.
2. The thread is started using start() method.
3. The run() method contains the task performed by the thread.
4. Multiple threads run concurrently under control of thread scheduler.

Example

```
class MyThread extends Thread {
public void run() {
System.out.println("Thread is running");
```

```
}
}

public class Test {
public static void main(String[] args) {
MyThread t1 = new MyThread();
t1.start();
}
}
```

Advantages

1. Better CPU utilization.
2. Faster execution of tasks.
3. Improves application responsiveness.
4. Useful for background operations.
5. Efficient resource sharing.

Disadvantages

1. Difficult to debug.
2. Complex program design.
3. Risk of deadlock and race condition.
4. Requires synchronization for shared data.

Q2. Explain Life Cycle of a Thread.

Introduction

A thread in Java passes through various states from its creation to termination. These states together form the life cycle of a thread. Understanding the life cycle is important to control thread execution and avoid errors in multithreaded programs.

Explanation

A thread goes through the following states:

1. New – Thread object is created but not started.
2. Runnable – start() method is called; thread is ready to run.
3. Running – Thread scheduler selects the thread for execution.
4. Blocked/Waiting – Thread waits for resources or another thread.
5. Terminated – Execution of run() method is completed.

Example

Thread t = new Thread();
t.start();

Advantages

1. Helps in thread management.
2. Useful in debugging.
3. Provides better control over execution.

Disadvantages

1. Complex state transitions.
2. Difficult to trace runtime behavior.

Q3. Explain Creating a Thread in Java.

Introduction

Threads in Java can be created using two methods. Java provides built-in support for thread creation through Thread class and Runnable interface.

Explanation

1. Extending Thread class
2. Implementing Runnable interface

Example

```
class MyRunnable implements Runnable {
public void run() {
System.out.println("Thread running");
}
}
```

Advantages

1. Supports multitasking.
2. Improves performance.
3. Easy implementation.

Disadvantages

1. Complex synchronization needed.
2. Hard debugging.

Diagram: Not required

Q4. Explain Thread Scheduler.

Introduction

Thread scheduler is a component of JVM that decides which thread will execute at a given time. Java uses priority-based scheduling.

Explanation

1. Each thread has priority (1–10).
2. Higher priority thread gets preference.
3. Scheduling depends on JVM and OS.

Example

```
Thread t1 = new Thread();
t1.setPriority(10);
```

Advantages

1. Efficient CPU use.
2. Priority control.
3. Supports critical task execution.

Disadvantages

1. Platform dependent.
2. May cause starvation.

Diagram: Not required

Q5. Explain Sleeping a Thread.

Introduction

sleep() method pauses execution of current thread for specified time.

Explanation

1. Thread goes to waiting state.
2. After time expires, returns to runnable state.
3. Does not release lock.

Example

```
try {
Thread.sleep(1000);
} catch(Exception e) {}
```

Advantages

1. Time control.
2. Useful in animations.
3. Prevents CPU overuse.

Disadvantages

1. Throws exception.
2. Does not release lock.

Q6. Explain Joining a Thread.

Introduction

join() method allows one thread to wait for another thread to finish execution.

Explanation

1. Main thread waits until child thread completes.
2. Used for dependent tasks.

Example

```
t1.start();
t1.join();
```

Advantages

1. Ensures correct sequence.
2. Useful in result calculation.
3. Prevents premature termination.

Disadvantages

1. Can cause delay.
2. May block main thread.

Diagram: Not required

Q7. Explain Thread Priority.

Introduction

Thread priority determines execution preference.

Explanation

1. Range: 1–10
2. Default priority: 5
3. Higher priority gets more CPU time.

Example

```
t1.setPriority(Thread.MAX_PRIORITY);
```

Advantages

1. Execution control.
2. Efficient scheduling.
3. Better resource management.

Disadvantages

1. Not guaranteed execution order.
2. Starvation possible.

Q8. Explain Thread Synchronization.

Introduction

Synchronization is used to control access to shared resources by multiple threads to avoid data inconsistency.

Explanation

1. Only one thread can access synchronized method at a time.
2. Prevents race condition.
3. Uses synchronized keyword.

Example

```
synchronized void display() { }
```

Advantages

1. Ensures data consistency.
2. Thread safety.
3. Prevents race condition.

Disadvantages

1. Reduces performance.
2. May cause deadlock.

Q9. Explain wait() Method.

Introduction

wait() method causes current thread to release lock and enter waiting state.

Explanation

1. Must be inside synchronized block.
2. Releases lock.
3. Waits until notify() or notifyAll() called.

Example

```
synchronized(obj) {
obj.wait();
}
```

Advantages

1. Efficient communication.
2. Prevents busy waiting.
3. Saves CPU cycles.

Disadvantages

1. Risk of indefinite waiting.
2. Complex implementation.

Q10. Explain notify() Method.

Introduction

notify() wakes up one waiting thread.

Explanation

1. Used inside synchronized block.
2. Only one thread resumes.

Advantages

1. Efficient signaling.
2. Improves coordination.
3. Reduces CPU usage.

Disadvantages

1. May cause starvation.
2. Uncertain thread selection.

Q11. Explain notifyAll() Method.

Introduction

notifyAll() wakes up all waiting threads.

Explanation

1. All waiting threads become runnable.
2. One gets lock at a time.

Advantages

1. Avoids starvation.
2. Better fairness.
3. Useful for multiple waiting threads.

Disadvantages

1. Performance overhead.
2. Lock competition.

Q12. Differentiate Between Process and Thread in Java.

Introduction

In an operating system, a process is an independent program in execution that has its own memory space, resources, and execution environment. Each process runs separately and does not share memory with other processes. A thread, on the other hand, is the smallest unit of execution within a process. Threads are also called lightweight processes because they share the same memory and resources of the parent process. Java supports multithreading, which allows multiple threads to run concurrently within a single process. Understanding the difference between process and thread is essential for writing efficient and optimized programs.

Explanation

1. A process has its own separate memory space, while threads share the same memory space.
2. Process creation is slow and resource-consuming, while thread creation is faster.
3. Context switching between processes is slower compared to threads.
4. Communication between processes is complex, but threads communicate easily through shared memory.

Example

When you open MS Word, it is a process. Spell check and auto-save inside it work as threads.

Advantages of Threads

1. Less memory usage.
2. Faster execution.
3. Easy communication.

Disadvantages

1. Shared memory may cause synchronization issues.
2. Errors in one thread can affect entire process.

Q13. What is Race Condition? Explain with Example.

Introduction

Race condition is a common problem in multithreading that occurs when two or more threads access shared data simultaneously and try to modify it without proper synchronization. Since threads execute concurrently, the final result depends on the order in which the threads are executed. This unpredictable behavior can lead to incorrect results and data inconsistency. Race conditions are difficult to detect and debug because they may not occur every time the program runs.

Explanation

If multiple threads update the same variable at the same time, they may overwrite each other's values.

Example

Suppose two threads increment a shared variable count = 0.
Both read value 0, increment to 1, and store 1.
Final value becomes 1 instead of 2.

Advantages of Understanding Race Condition

1. Helps in writing safe multithreaded programs.
2. Improves reliability.
3. Prevents data corruption.

Disadvantages

1. Hard to detect.
2. Causes unpredictable errors.
3. Debugging is complex.

Q14. What is Deadlock? Explain its Causes and Prevention.

Introduction

Deadlock is a serious problem in multithreading where two or more threads are blocked forever because each thread is waiting for a resource that is held by another thread. In such a situation, none of the threads can proceed, and the program becomes stuck. Deadlock generally occurs in systems where multiple threads compete for limited resources such as files, memory, or database connections. Understanding deadlock is important to design safe and stable concurrent systems.

Explanation

Deadlock occurs when four conditions are present:

1. Mutual Exclusion – Resource cannot be shared.
2. Hold and Wait – Thread holds one resource and waits for another.
3. No Preemption – Resource cannot be forcibly taken.
4. Circular Wait – Threads wait in circular chain.

Example

Thread 1 locks Resource A and waits for Resource B.
Thread 2 locks Resource B and waits for Resource A.

Prevention

1. Acquire resources in fixed order.
2. Avoid nested locks.
3. Use timeout mechanism.

Advantages of Studying Deadlock

1. Prevents system crash.
2. Improves program stability.

Disadvantages

1. Hard to detect.
2. Can freeze application completely.

Q15. Explain Inter-thread Communication in Java.

Introduction

Inter-thread communication is a mechanism that allows threads to communicate and coordinate with each other efficiently. In multithreaded programs, threads often depend on each other to complete tasks. Instead of continuously checking a condition (busy waiting), Java provides methods such as wait(), notify(), and notifyAll() to enable proper communication between threads. These methods are defined in the Object class and must be used within synchronized blocks.

Explanation

1. wait() – Causes the current thread to release the lock and wait.
2. notify() – Wakes up one waiting thread.
3. notifyAll() – Wakes up all waiting threads.

Example: Used in Producer–Consumer problem.

Advantages

1. Efficient CPU utilization.
2. Proper coordination between threads.
3. Avoids unnecessary looping.

Disadvantages

1. Complex implementation.
2. Risk of deadlock if misused.

Q16. Differentiate Between Synchronized Method and Synchronized Block in Java.

Introduction

In multithreading, synchronization is used to control access to shared resources and prevent data inconsistency. When multiple threads try to access the same resource simultaneously, it may lead to race conditions. To solve this problem, Java provides the synchronized keyword. Synchronization can be applied either to an entire method (synchronized method) or to a specific portion of code (synchronized block). Understanding the difference between synchronized method and synchronized block is important for writing efficient and safe multithreaded programs.

Explanation

Synchronized Method

1. Entire method is locked.
2. Only one thread can execute the method at a time.
3. Lock is applied on the object.

Example

```
synchronized void display() {
System.out.println("Hello");
}
```

Synchronized Block

1. Only specific part of method is locked.
2. Improves performance because smaller portion is synchronized.
3. Lock can be applied on any object.

Example

```
void display() {
synchronized(this) {
System.out.println("Hello");
}
}
```

Advantages

1. Ensures data consistency.

2. Prevents race condition.
3. Improves thread safety.

Disadvantages

1. Reduces performance if overused.
2. May cause deadlock.
3. Increases program complexity.

Q17. What is a Daemon Thread in Java? Explain with Example.

Introduction

A daemon thread is a background thread that runs to support user threads. It provides services such as garbage collection, background monitoring, or housekeeping tasks. The main characteristic of a daemon thread is that it automatically terminates when all user threads finish execution. It does not prevent the JVM from exiting. Daemon threads are mainly used for background tasks that do not require continuous execution once the main task is complete.

Explanation

1. Runs in background.
2. Automatically stops when user threads finish.
3. Must be set before starting the thread using setDaemon(true).

Example

```
Thread t1 = new Thread();
t1.setDaemon(true);
t1.start();
```

Advantages

1. Useful for background services.
2. Automatic termination.
3. Reduces manual resource management.

Disadvantages

1. May terminate abruptly.
2. Not suitable for important tasks.

3. No guarantee of complete execution.

Q18. Explain Thread States in Detail.

Introduction

A thread in Java goes through several states during its life cycle. These states represent the different stages of execution of a thread from creation to termination. Understanding thread states helps in debugging, synchronization, and managing multithreaded programs effectively. Java provides the Thread.State enum to represent different thread states.

Explanation

The main thread states are:

1. NEW – Thread object created but not started.
2. RUNNABLE – Thread is ready to run.
3. BLOCKED – Waiting for monitor lock.
4. WAITING – Waiting indefinitely for another thread's action.
5. TIMED_WAITING – Waiting for specified time (sleep).
6. TERMINATED – Execution completed.

Each state represents a specific condition of thread during execution.

Advantages

1. Helps in debugging thread behavior.
2. Provides better control over execution.
3. Improves program design.

Disadvantages

1. Complex state transitions.
2. Difficult to track in large programs.

Q19. What are the Advantages of Multithreading in Java? Explain in Detail.

Introduction

Multithreading is one of the most powerful features of Java that allows multiple threads to execute concurrently within a single program. It improves application performance, responsiveness, and resource utilization. Modern applications such as web servers, banking systems, and real-time systems heavily rely on multithreading to handle multiple tasks simultaneously.

Explanation

1. Threads run concurrently within same process.
2. Share memory and resources.
3. Improve execution speed.

Advantages

1. Improved performance.
2. Better CPU utilization.
3. Faster context switching.
4. Efficient resource sharing.
5. Improved user experience.

Disadvantages

1. Complex programming.
2. Debugging difficulty.
3. Risk of synchronization issues.
4. Possible deadlock and starvation.

Q20. What are the Common Problems in Multithreading? Explain with Examples.

Introduction

Although multithreading increases efficiency and performance, it also introduces several challenges due to concurrent execution. When multiple threads share resources, improper handling can lead to serious problems such as race conditions, deadlocks, and starvation. These problems make multithreaded programming more complex than single-threaded programming.

Explanation

1. Race Condition – Multiple threads modify shared data simultaneously.

2. Deadlock – Threads wait indefinitely for each other's resources.
3. Starvation – Low-priority thread never gets CPU time.
4. Livelock – Threads change state but do not progress.
5. Thread Interference – Incorrect results due to unsynchronized access.

Solutions

1. Use synchronized keyword.
2. Use proper locking mechanism.
3. Avoid circular waiting.
4. Minimize shared resources.

Advantages of Studying These Problems

1. Helps in writing safe programs.
2. Improves reliability.
3. Prevents runtime failures.

Disadvantages

1. Hard to detect errors.
2. Complex debugging process.